

ASSIGNMENT 2 REPORT

RAHUL VIMALKANTH
EE23B061

September 7, 2024

1 Introduction

This report outlines the methodology for analyzing electrical circuits using matrix methods. The approach involves parsing a SPICE-like circuit file, constructing matrices based on Kirchhoff's laws, and solving the resulting linear system to determine node voltages and currents through voltage sources.

2 Function Description

2.1 Parsing the File

- **File Validation:** Checks if the file exists and is not empty.
- **File Reading:** Reads the content of the file and extracts circuit components. Lines between '.circuit' and '.end' are considered valid circuit components.
- **Component Parsing:** Identifies and extracts resistors ('R'), voltage sources ('V'), and current sources ('I').
- Comments are removed, and valid lines are parsed into component lists.

2.2 Node Mapping

- **Initialization:** The node mapping starts with **GND** set to index 0. A counter (**node_index**) is initialized to 1, which will be incremented for each new node encountered.
- **Mapping Nodes:** As the components are parsed, the nodes connected by each component are extracted. If a node has not been previously mapped, it is assigned the next available index, and the **node_index** counter is incremented.
- **Node Storage:** The mapping of nodes to indices is stored in a dictionary (**node_map**), where keys are node names and values are their corresponding indices.

2.3 Component Storage

Components such as resistors, voltage sources, and current sources are stored in a list called `mod_components`. Each component is parsed and its details are stored in this list for further processing.

- **Component Parsing:** Each line of the circuit file between the `.circuit` and `.end` markers is considered a valid component. Components are identified by their prefixes: ‘R’ for resistors, ‘V’ for voltage sources, and ‘I’ for current sources.
- **Storing Components:** Each component is parsed into a list where the first element is the component identifier, and the subsequent elements are the nodes it connects to. These lists are then appended to the `mod_components` list.
- **Node Identification:** During the parsing of each component, the nodes it connects to are identified and added to a set (`nodes`). This set ensures that all nodes are captured, even if they are connected to multiple components.

2.4 Kirchhoff’s Laws

- **Kirchhoff’s Current Law (KCL):** Ensures that the sum of currents entering and leaving a node equals zero. This is reflected in the conductance matrix G .
- **Kirchhoff’s Voltage Law (KVL):** Ensures that the sum of voltages around any closed loop equals zero. Constraints from voltage sources are added to the matrix and vector.

2.5 Matrix Size Calculation

After all nodes and components have been mapped and stored, the size of the conductance matrix is determined. The matrix size depends on the number of nodes and the number of voltage sources in the circuit.

- **Number of Nodes:** The total number of nodes is given by the size of the `node_map` dictionary. However, since `GND` is always at index 0 and does not contribute to the matrix size, the number of nodes used in the matrix is `num_nodes - 1`.
- **Number of Voltage Sources:** The number of voltage sources is determined by counting the number of components in `mod_components` that start with ‘V’.
- **Matrix Size:** The total size of the matrix M is calculated as the number of nodes plus the number of voltage sources minus one, i.e., $M = \text{num_nodes} + \text{num_vsources} - 1$.

2.6 Matrix Construction

- **Conductance Matrix (G):** For resistors, conductance values are added to diagonal elements and subtracted from off-diagonal elements. For voltage sources, additional rows and columns are added to handle the constraints imposed by the sources.
- **Current Vector (I):** Current sources directly modify the vector. Voltage sources modify the vector based on their voltage values and constraints imposed on the matrix.
- **System of Equations:** Represents the circuit equations as $G\mathbf{x} = \mathbf{I}$.
- **Solving:** Uses `numpy.linalg.solve` to solve the system and obtain node voltages and currents through voltage sources.

2.7 Handling Voltage Sources

When processing voltage sources, special attention is needed to integrate their effects into the conductance matrix and current vector. The detailed logic is as follows:

- **Identification of Voltage Source:** For each voltage source, identify the nodes $n1$ and $n2$ between which the source is connected. The `v_index` tracks the position of the voltage source in the matrix.
- **Current Vector Update:** Set the entry in the current vector I at position `v_index` to the voltage source value. This represents the effect of the voltage source on the nodes.
- **Index Management:** Increment the `v_index` after processing each voltage source to ensure that each source has a unique position in the matrix and vector.

2.8 Error Handling

- **File Errors:** Checks for valid file input.
- **Malformed Input:** Validates that components follow correct SPICE syntax and contain necessary parameters.
- **Unsolvable Systems:** Handles scenarios where the matrix is singular or otherwise unsolvable, raising a `ValueError`.

3 Extra Tests Implemented

- **Valid Input:** Tests with well-defined circuits to verify correctness.
- **Invalid Input:** Tests for malformed files and unexpected inputs.
- **Presence of Capacitor:** Tests to verify that the code identifies and rejects capacitors, which are not handled in the current implementation.
- **Multiple Voltage Sources:** Verifies that the function correctly processes multiple voltage sources in different configurations, such as parallel and series.
- **No GND Node:** Ensures that the circuit definition includes a ground node and raises an error if it is missing.
- **Non-numeric Component Values:** Checks for invalid component values that are not numerical, ensuring that they are detected and flagged.
- **Open Circuit:** Tests for open circuits where components are not fully connected to the rest of the circuit.
- **Short Circuit:** Identifies and handles cases where two nodes are directly connected without any resistance or source.

—Thank You—